

System Design Document

1. Architecture Overview

EVOLVE is designed as a modular, frontend-only Single Page Application (SPA). It uses a custom component-based architecture and a centralized, reactive state management pattern, removing the need for external frontend frameworks like React or Vue.

Tech Stack

* **Core:** HTML5, Vanilla JavaScript (ES Modules), CSS3

* **Build Tool:** Vite

* **Libraries:** Chart.js (for analytics/visualization)

2. High-Level Component Design

The system is divided into logical modules: core application shell, state management, algorithmic utilities, data mocks, and UI components.

2.1 UI Components (`src/components/`)

Each component is an ES6 class that receives a DOM container and implements `init()` and `render()` methods.

* `Dashboard.js` : Main landing view displaying metrics and charts.

* `Kanban.js` : Drag-and-drop board managing CR lifecycle statuses.

* `ImpactAnalysis.js` : Visualizes the dependency graph and risk scores for a selected CR.

* `Traceability.js` : Shows inline code diffs for affected components.

* `Header.js` : Global navigation and global action triggers (e.g., "Create CR").

2.2 State Management (`src/utls/store.js`)

Implements a centralized **StateStore** using a Publish-Subscribe (Pub/Sub) pattern.

* **State Tree:** Stores `activeView`, `changeRequests`, `codebase` metadata, and current selection contexts (`selectedComponentId`, `activeImpactCR`).

* **Reactivity:** Components subscribe to the store. When state mutates via actions (e.g., `addChangeRequest`, `updateCRStatus`), the store notifies all subscribers, triggering re-renders.

2.3 Impact Engine (`src/utils/impactEngine.js`)

Contains the core business logic for change propagation.

* **Graph Traversal:** Uses Breadth-First Search (BFS) on an inverted dependency graph (propagation graph).

* **Risk Calculation:** Calculates an evolution risk score based on traversal distance (direct vs transitive), component complexity, and churn rate.

3. Data Flow

1. **User Action:** User interacts with the UI (e.g., drops a CR card into a new Kanban column).
2. **State Update:** The UI component calls a store action (e.g., `store.updateCRStatus(id, newStatus)`).
3. **Mutation:** The store updates the internal state tree.
4. **Notification:** The store calls `notify()`, which invokes subscriber callbacks.
5. **Re-render:** The `App` controller (in `main.js`) receives the state change and re-renders the active component with the fresh data.

4. UI/UX Design System

* **Styling:** Controlled globally via CSS variables in `variables.css`, implementing a coherent theme (colors, spacing, fonts).

* **Typography:** Integrates modern web fonts (Outfit, JetBrains Mono, Plus Jakarta Sans) for varied text hierarchies.